$ PHPUNIT --TESTDOX

PHPUnit: Beyond Assertions

Dan Soppelsa · Longhorn PHP · Austin, TX · 2026

https://joind.in/talk/ec1aa

$ WHOAMI

Dan Soppelsa

> Web Developer @ Vehikl

> @dansoppelsa

$ CAT /ETC/EMPLOYER

VEHIKL

Design · Develop · Deliver

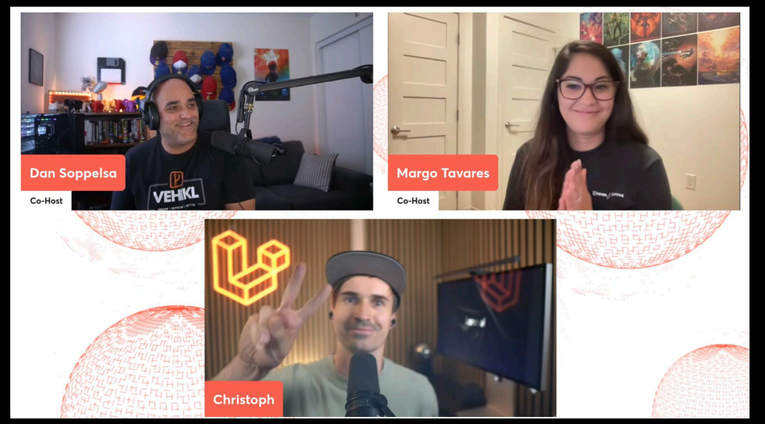
Laracon 2026 · Boston

$ CD ~/COMMUNITY

Laravel Worldwide Meetup

meetup.laravel.com

youtube.com/laravelphp



podcast.vehikl.com

Why Do We Write Tests?

Why Do We Write Tests?

> Instant Validation
> Spot ambiguities early
> Safe Refactoring
> Better Design
> Living Documentation
> Long-Term Time Savings
> Catch Regression Early
> Guard Against Regressions
> Release with more confidence

Tests --About

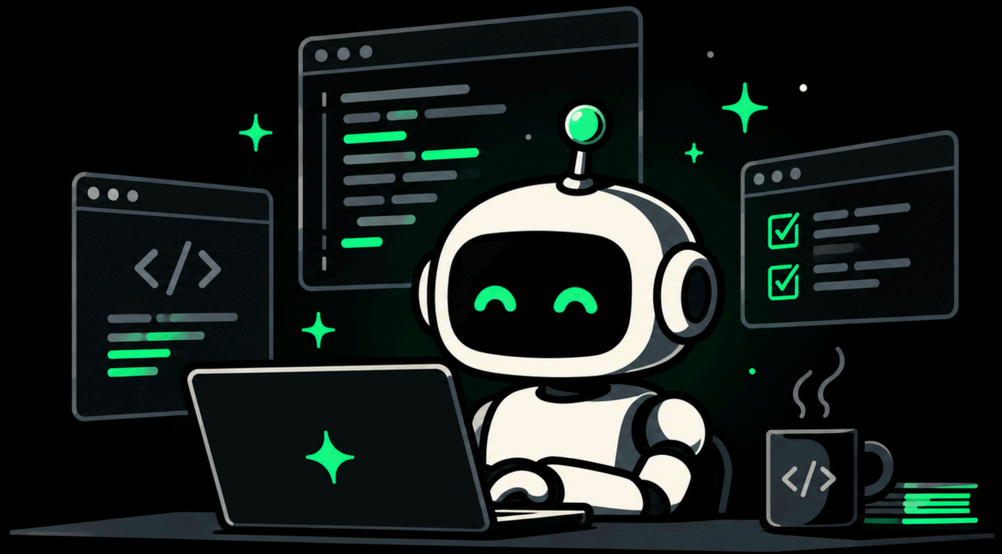
> TESTS ARE CODE.

Let’s Treat Them Like It.

> TESTS ARE TOOLS.

Let’s Make Them Work For Us.

Tests --About



> BUT DO TESTS STILL MATTER?

In The Age of Agents?

nunomaduro
@enunomaduro

the importance of tests in 2026 is a thousand times bigger than before, literally

we're treating software more like a black box, still review the code, but we care more and more about input and output, and less about what's inside.. making sure the output stays the same no matter the input is more important than ever

it's a crucial skill to have:

- browser testing
- feature testing
- unit testing
- integration testing

test it all the way through, 100% code coverage and more

9:49 AM · Sep 27, 2026 · **15K** Views

What Do You Want From Your Test Suite?

WHAT DO YOU WANT
...FROM YOUR TEST SUITE?
imgflip.com

Test Desiderata

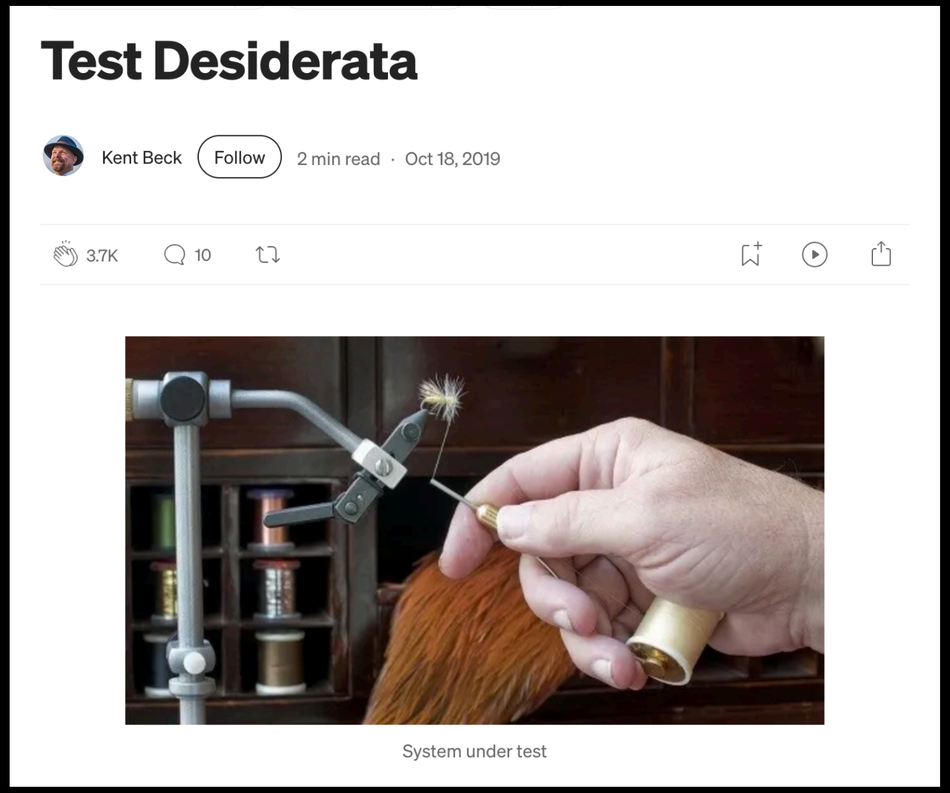
Test Desiderata

PLURAL NOUN

desiderata

things wanted or needed; the plural of desideratum.

Test Desiderata



https://medium.com/@kentbeck_7670/test-desiderata-94150638a4b3

Test Desiderata

Test Desiderata

> *I thought this property of tests was obvious and widely understood…*
>
> *The experience got me thinking about the other properties of tests that I take for granted. Here, then, is a list of properties of tests. Not all tests need to exhibit all properties. However, no property should be given up without receiving a property of greater value in return.*

- Kent Beck

Test Desiderata

Test Desiderata

> Isolated
> Composable
> Fast
> Inspiring
> Writable
> Readable
> Behavioral
> Structure-insensitive
> Automated
> Specific
> Deterministic
> Predictive

Test Desiderata

> Isolated

tests should return the same results regardless of the order in which they are run.

> Composable

if tests are isolated, then I can run 1 or 10 or 100 or 1,000,000 and get the same results.

> Fast

tests should run quickly.

> Inspiring

passing the tests should inspire confidence

Test Desiderata

> Writable

tests should be cheap to write relative to the cost of the code being tested.

> Readable

tests should be comprehensible for reader, invoking the motivation for writing this particular test.

> Behavioral

tests should be sensitive to changes in the behavior of the code under test. If the behavior changes, the test result should change.

> Structure-insensitive

tests should not change their result if the structure of the code changes.

Test Desiderata

> Automated

tests should run without human intervention.

> Specific

if a test fails, the cause of the failure should be obvious.

> Deterministic

if nothing changes, the test result shouldn’t change.

> Predictive

if the tests all pass, then the code under test should be suitable for production.

Test Desiderata

> Isolated
> Composable
> Fast
> Inspiring
> Writable
> Readable
> Behavioral
> Structure-insensitive
> Automated
> Specific
> Deterministic
> Predictive

Test Desiderata

> Writable
> Readable
> Behavioral
> Specific

PHPUnit: Beyond Assertions

> Test Setup

> Custom Assertions

> Custom Constraints

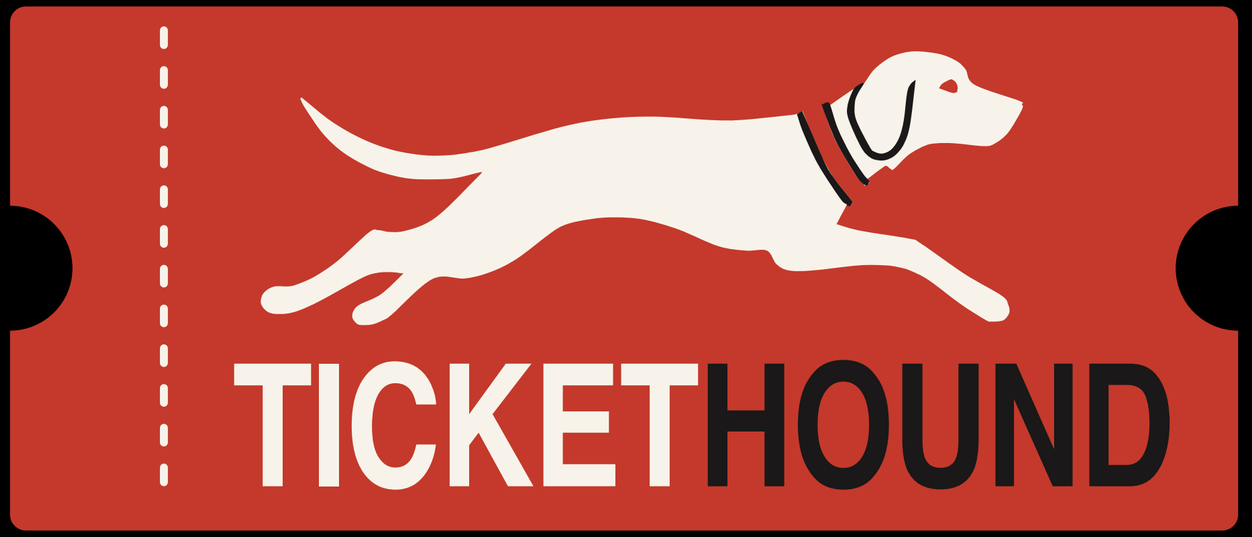
> Custom Comparators

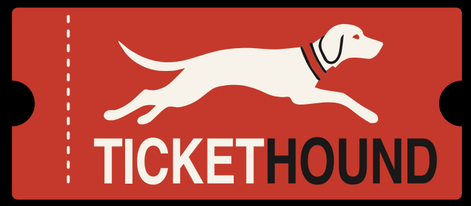
> Custom Object Exporters

PHPUnit: Beyond Assertions

> Friendly

> Friendly

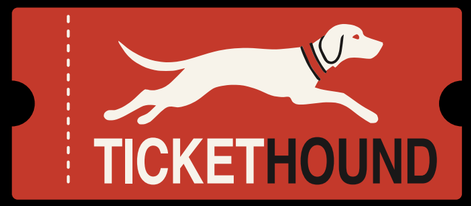

TICKETHOUND



2015

It started with one venue.

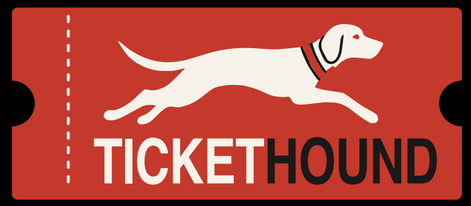
A small PHP application for an independent music venue.



2017

One venue became many.

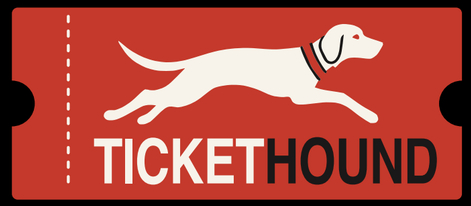
TicketHound was born.



2020

An interesting year to work for a ticketing company.

~~Cancelled~~ ~~Postponed~~ ~~Rescheduled~~ ~~Refunded~~



Today

11 years (77 dog years) of application code.

Many tests, Many Developers

TICKETHOUND

Live music Comedy Theatre Festivals Community

Sign in

TICKETING FOR INDEPENDENT VENUES

SNIFF OUT YOUR NEXT EVENT

900+ venues, 1 Box Office

ARTIST, VENUE OR EVENT: Lakeshore Sunset Club

CITY: Austin

WHEN: This week

SEARCH

NT-LINE 24/7 PARADIGM · KEMMER-STAMM ROOM · SUN 9:30PM + SECURED ASYMMETRIC PORTAL · O'CONNELL AND SONS CLUB · FRI 7:30PM + VISIONARY BACKGROUND SUCCESS · ERDMAN-TRANTOW THEATRE · FRI 9:00PM + PE

1.2M TICKETS ISSUED

910 VENUES

10 MINUTE TICKET GUARANTEE

77 DOG YEARS OF SERVICE

MOVING FASTEST THIS WEEK

POPULAR EVENTS

Reactive Bifurcated Firmware

POLLICH GROUP THEATRE · SEATTLE

From $15

Business-Focused Neutral Moratorium

KREIGER-WILKINSON ROOM · CHICAGO

From $18

Synchronised Fresh-Thinking Website

STOKES PLC STAGE · TORONTO

From $17

TicketHound
LIVE
LOCAL
TOGETHER
COLD SNAP
COMEDY GALA
THE LANTERN ROOM
Cold Snap Comedy Gala
The Lantern Room
TICKETS
General Admission
QUANTITY
2
GENERAL ADMISSION
GENERAL ADMISSION
CHOOSE TICKETS
MUSIC
COMEDY
COMMUNITY

TicketHound
LIVE
LOCAL
TOGETHER
COLD SNAP
COMEDY GALA
THE LANTERN ROOM
Checkout
Complete your purchase to lock in your tickets.
YOUR TICKETS ARE HELD FOR
9:26
Payment Details
Secure and encrypted
Card Number
1234 5678 9012 3456
Expiration Date
MM / YY
CVC
123
Name on Card
Jane Doe
Complete Purchase →
Order Summary
2 × General Admission $80.00
Service Fee $8.00
Total (CAD) $88.00
Your tickets are held but not yet purchased.
Sat, Jan 24, 2026
8:00 PM
The Lantern Room
Kitchener, ON
MUSIC
COMEDY
COMMUNITY

TicketHound
LIVE
LOCAL
TOGETHER
YOU'RE GOING TO
COLD SNAP COMEDY GALA!
Your tickets are confirmed.
COLD SNAP
COMEDY GALA
THE LANTERN ROOM
Sat, Jan 24, 2026
8:00 PM
The Lantern Room
Kitchener, ON
2 × General Admission
Tickets sent to your email
View My Tickets →
Back to Events
MUSIC
COMEDY
COMMUNITY

TicketHound
LIVE
LOCAL
TOGETHER
COLD SNAP
COMEDY GALA
THE LANTERN ROOM
Sat, Jan 24, 2026
8:00 PM
The Lantern Room
Kitchener, ON
0:00
SORRY, YOUR CHECKOUT
HAS EXPIRED
Your ticket reservation could no longer be held.
Please re-select tickets and try again.
Browse Events
MUSIC
COMEDY
COMMUNITY

App\BoxOffice;

```php
class BoxOffice
{
    public function hold(Customer $customer, Event $event, SeatRequest ...$seats): Reservation;

    public function confirm(Reservation $reservation);
}
```

BoxOfficeTest

```
> holding_seats_creates_a_ten_minute_reservation();
> holding_seats_reduces_remaining_capacity();
> cannot_hold_more_seats_than_remain();
> expired_holds_release_inventory();
> confirming_a_hold_creates_a_paid_order_with_tickets();
> confirming_mixed_ticket_types_issues_the_right_counts();
> cannot_confirm_an_expired_hold();
> cannot_confirm_a_reservation_twice();
```

Test Phases

01 **Arrange** — Configure the state of “the world” required for the test

02 **Act** — Take the action you are testing the behavior of

03 **Assert** — Verify that the action produced the expected outcome

BoxOfficeTest > creating a reservation

```
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{









}
```

BoxOfficeTest > creating a reservation

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame('2026-09-08 19:10:00', $reservation->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

BoxOfficeTest > creating a reservation

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame('2026-09-08 19:10:00', $reservation->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

BoxOfficeTest > creating a reservation

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame('2026-09-08 19:10:00', $reservation->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

BoxOfficeTest > creating a reservation

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame('2026-09-08 19:10:00', $reservation->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

BoxOfficeTest > confirming a hold

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
```

BoxOfficeTest > confirming a hold

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
```

BoxOfficeTest > confirming a hold

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
```

BoxOfficeTest > confirming a hold

```
            ->forEvent($event)
            ->price(2000)
            ->create(['capacity' => 50, 'name' => 'General Admission']);
        $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

        $order = $this->boxOffice()->confirm($reservation);

        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
        $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
        $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertSame($order->total_amount, $order->payments->first()->amount);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertCount(2, $order->tickets);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
        $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
        $this->assertSame(48, $ga->remaining());
    }
```

BoxOfficeTest > handling an expired hold

```php
#[Test]
public function cannot_confirm_an_expired_hold(): void
{









}
```

BoxOfficeTest > handling an expired hold

```php
#[Test]
public function cannot_confirm_an_expired_hold(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    Carbon::setTestNow('2026-09-08 19:10:00');

    try {
        $this->boxOffice()->confirm($reservation);
        $this->fail('Expected ReservationExpired to be thrown.');
    } catch (ReservationExpired) {
        $this->assertSame(ReservationStatus::EXPIRED, $reservation->fresh()->status);
        $this->assertSame(0, Order::query()->count());
        $this->assertSame(50, $ga->remaining());
    }
}
```

BoxOfficeTest > handling an expired hold

```php
#[Test]
public function cannot_confirm_an_expired_hold(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    Carbon::setTestNow('2026-09-08 19:10:00');

    try {
        $this->boxOffice()->confirm($reservation);
        $this->fail('Expected ReservationExpired to be thrown.');
    } catch (ReservationExpired) {
        $this->assertSame(ReservationStatus::EXPIRED, $reservation->fresh()->status);
        $this->assertSame(0, Order::query()->count());
        $this->assertSame(50, $ga->remaining());
    }
}
```

BoxOfficeTest > handling an expired hold

```php
#[Test]
public function cannot_confirm_an_expired_hold(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    Carbon::setTestNow('2026-09-08 19:10:00');

    try {
        $this->boxOffice()->confirm($reservation);
        $this->fail('Expected ReservationExpired to be thrown.');
    } catch (ReservationExpired) {
        $this->assertSame(ReservationStatus::EXPIRED, $reservation->fresh()->status);
        $this->assertSame(0, Order::query()->count());
        $this->assertSame(50, $ga->remaining());
    }
}
```

BoxOfficeTest > purchasing mixed ticket types

```php
#[Test]
public function confirming_mixed_ticket_types_issues_the_right_counts(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $reserved = TicketType::factory()
        ->for($event)
        ->price(8000)
        ->create(['capacity' => 10, 'name' => 'Reserved Seating']);
    $reservation = $this->boxOffice()->hold(
        $customer,
        $event,
        new SeatRequest($ga, 2),
        new SeatRequest($reserved, 1),
    );

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
```

BoxOfficeTest > purchasing mixed ticket types

```php
#[Test]
public function confirming_mixed_ticket_types_issues_the_right_counts(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $reserved = TicketType::factory()
        ->for($event)
        ->price(8000)
        ->create(['capacity' => 10, 'name' => 'Reserved Seating']);
    $reservation = $this->boxOffice()->hold(
        $customer,
        $event,
        new SeatRequest($ga, 2),
        new SeatRequest($reserved, 1),
    );

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
```

BoxOfficeTest > purchasing mixed ticket types

```
    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(12000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(750, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(12750, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(3, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSU
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(2, $order->tickets->where('ticket_type_id', $ga->id)->count());
    $this->assertSame(1, $order->tickets->where('ticket_type_id', $reserved->id)->count());
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
    $this->assertSame(9, $reserved->remaining());
}
```

BoxOfficeTest > the whole enchilada

```php
<?php

namespace Tests\Feature;
```

THIS IS
FINE

No, This Actually Is Fine

BoxOfficeTest

> deterministic
> predictive
> specific
> cover the behavio(u)r of mission critical code paths
> alert other developers when a regression is introduced
> inspiring

THIS IS
FINE

Test Desiderata
Writable
Readable
Behavioral
Specific

Test Setup

Test Setup

> Visually Group Test Phases

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());
    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());
    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Visually Group Test Phases

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());

    Carbon::setTestNow('2026-09-08 19:10:00');

    $this->assertSame(50, $ga->remaining());
    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup

> Avoid Red Herrings

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow('2026-09-08 19:10:00');
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow('2026-09-08 19:10:00');
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow('2026-09-08 19:10:00');
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow('2026-09-08 19:10:00');
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow('2026-09-08 19:10:00');
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame('2026-09-08 19:20:00', $laterHold->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow(Carbon::now()->addWeeks(2));
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow(Carbon::now()->addMinutes(10));
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow(Carbon::now()->addWeeks(2));
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow(Carbon::now()->addMinutes(10));
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    Carbon::setTestNow(Carbon::now()->addWeeks(2));
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    Carbon::setTestNow(Carbon::now()->addMinutes(10));
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    $this->travel(10)->minutes();
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    $this->travel(10)->minutes();
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Avoid Red Herrings

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    $this->travel(10)->minutes();
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup

> Centralize Shared Test Preparation

Test Setup > Centralize Shared Setup

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->create(['capacity' => 50]);
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    $this->travel(10)->minutes();
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->price(2000)->create();

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $reservation->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertSame(2000, $reservation->items->first()->unit_price_amount);
    $this->assertSame('USD', $reservation->items->first()->unit_price_currency);
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function holding_seats_reduces_remaining_capacity(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->capacity(50)->create();

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Centralize Shared Setup

```php
protected Customer $customer;
protected Event $event;

public function setUp(): void
{
    parent::setUp();
    $this->customer = Customer::factory()->create();
    $this->event = Event::factory()->create();
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->capacity(50)->create();
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    $this->travel(10)->minutes();
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function expired_holds_release_inventory(): void
{
    $this->freezeTime();
    $ga = TicketType::factory()->for($event)->capacity(50)->create();
    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));
    $this->assertSame(48, $ga->remaining());
    $this->travel(10)->minutes();
    $this->assertSame(50, $ga->remaining());

    $laterHold = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $laterHold->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $laterHold->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $laterHold->items->sum('quantity'));
    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    $this->freezeTime();
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->price(2000)->create();

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $reservation->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    $this->freezeTime();
    $ga = TicketType::factory()->for($event)->price(2000)->create();

    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame(
        Carbon::now()->addMinutes(10)->format('Y-m-d H:i:s'),
        $reservation->expires_at->format('Y-m-d H:i:s')
    );
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function holding_seats_reduces_remaining_capacity(): void
{

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()->for($event)->capacity(50)->create();

    $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());
}
```

Test Setup > Centralize Shared Setup

```php
#[Test]
public function holding_seats_reduces_remaining_capacity(): void
{
    $ga = TicketType::factory()->for($event)->capacity(50)->create();

    $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());
}
```

```
➜  tickethound git:(main) ./vendor/bin/phpunit tests/Feature/BoxOfficeTest.php
```

#ShipIt

Custom Assertions

Custom Assertions

> Consolidate repeated, *generic* assertions about the object(s) under test

Custom Assertions

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Assertions

```
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Assertions

```
#[Test]
public function confirming_mixed_ticket_types_issues_the_right_counts(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reserved = TicketType::factory()->for($this->event)->price(8000)->capacity(10)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2), new
SeatRequest($reserved, 1));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(12000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(750, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(12750, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(3, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(2, $order->tickets->where('ticket_type_id', $ga->id)->count());
```

Custom Assertions

```
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2), new SeatRequest($reserved, 1));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(12000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(750, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(12750, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(3, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(2, $order->tickets->where('ticket_type_id', $ga->id)->count());
    $this->assertSame(1, $order->tickets->where('ticket_type_id', $reserved->id)->count());
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
    $this->assertSame(9, $reserved->remaining());
}
```

Custom Assertions

```php
#[Test]
public function cannot_confirm_a_reservation_twice(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));
    $order = $this->boxOffice()->confirm($reservation);

    try {
        $this->boxOffice()->confirm($reservation->fresh());

        $this->fail('Expected ReservationAlreadyConverted to be thrown.');
    } catch (ReservationAlreadyConverted) {
        $order = $order->fresh(['payments', 'tickets']);

        $this->assertSame(1, Order::query()->count());
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
        $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
        $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertSame($order->total_amount, $order->payments->first()->amount);
        $this->assertNotNull($order->payments->first()->paid_at);
```

Custom Assertions

```
            $this->fail('Expected ReservationAlreadyConverted to be thrown.');
        } catch (ReservationAlreadyConverted) {
            $order = $order->fresh(['payments', 'tickets']);

            $this->assertSame(1, Order::query()->count());
            $this->assertSame(OrderStatus::CONFIRMED, $order->status);
            $this->assertNotNull($order->confirmation_number);
            $this->assertNotNull($order->confirmed_at);
            $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
            $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
            $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
            $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
            $this->assertCount(1, $order->payments);
            $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
            $this->assertSame($order->total_amount, $order->payments->first()->amount);
            $this->assertNotNull($order->payments->first()->paid_at);
            $this->assertCount(2, $order->tickets);
            $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
    TicketStatus::ISSUED));
            $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
            $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
            $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
            $this->assertSame(48, $ga->remaining());
        }
    }
```

Custom Assertions

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $ticket): bool => $ticket->status ===
TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $ticket): bool => $ticket->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $ticket): bool => $ticket->code !== null));
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Assertions

```
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
```

Custom Assertions

```php
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
```

Custom Assertions

```
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

Custom Assertions

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertSame(4000, $order->subtotal_amount);
    $this->assertSame(500, $order->fee_amount);
    $this->assertSame(4500, $order->total_amount);
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Assertions

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertOrderIsConfirmed($order);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertCount(2, $order->tickets);
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Assertions

```
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2), new
SeatRequest($reserved, 1));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(12000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(750, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(12750, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(3, $order->tickets);
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status === TicketStatus::ISSUED));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
    $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    $this->assertSame(2, $order->tickets->where('ticket_type_id', $ga->id)->count());
    $this->assertSame(1, $order->tickets->where('ticket_type_id', $reserved->id)->count());
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
    $this->assertSame(9, $reserved->remaining());
}
```

Custom Assertions

```
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2), new
SeatRequest($reserved, 1));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertOrderIsConfirmed($order);
    $this->assertEquals(Money::ofMinor(12000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(750, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(12750, 'USD'), $order->total());
    $this->assertCount(3, $order->tickets);
    $this->assertSame(2, $order->tickets->where('ticket_type_id', $ga->id)->count());
    $this->assertSame(1, $order->tickets->where('ticket_type_id', $reserved->id)->count());
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
    $this->assertSame(9, $reserved->remaining());
}
```

Custom Assertions

```
        $this->fail('Expected ReservationAlreadyConverted to be thrown.');
    } catch (ReservationAlreadyConverted) {
        $order = $order->fresh(['payments', 'tickets']);

        $this->assertSame(1, Order::query()->count());
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
        $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
        $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertSame($order->total_amount, $order->payments->first()->amount);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertCount(2, $order->tickets);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
        $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
        $this->assertSame(48, $ga->remaining());
    }
}
```

Custom Assertions

```
        $this->fail('Expected ReservationAlreadyConverted to be thrown.');
    } catch (ReservationAlreadyConverted) {
        $order = $order->fresh(['payments', 'tickets']);

        $this->assertSame(1, Order::query()->count());
        $this->assertOrderIsConfirmed($order);
        $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
        $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
        $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
        $this->assertCount(2, $order->tickets);
        $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
        $this->assertSame(48, $ga->remaining());
    }
}
```

```
➜  tickethound git:(main) ./vendor/bin/phpunit tests/Feature/BoxOfficeTest.php
```

#ShipIt

Custom Constraints

A FEW
MOMENTS LATER

Custom Constraints

```
for ($i = 0; $i < $item->quantity; $i++) {
    $order->tickets()->create([
        'order_item_id' => $orderItem->id,
        'ticket_type_id' => $item->ticket_type_id,
        'code' => 'TK-'.strtoupper(Str::random(8)),
        'status' => TicketStatus::ISSUED,
        'issued_at' => now(),
    ]);
}
```

```
107  107
108  108  for ($i = 0; $i < $item->quantity; $i++) {
109  109      $order->tickets()->create([
110  110          'order_item_id' => $orderItem->id,
111  111          'ticket_type_id' => $item->ticket_type_id,
112  112          'code' => 'TK-'.strtoupper(Str::random(8)),
113  113          'status' => TicketStatus::ISSUED,
114  114      ]);
115  115  }
116  116
```

```
sail@e99c11416379:/var/www/html$
```

Custom Constraints

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Failed asserting that false is true.

/var/www/html/tests/BoxOfficeAssertions.php:25
/var/www/html/tests/Feature/BoxOfficeTest.php:114
```

Custom Constraints

```php
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

Custom Constraints

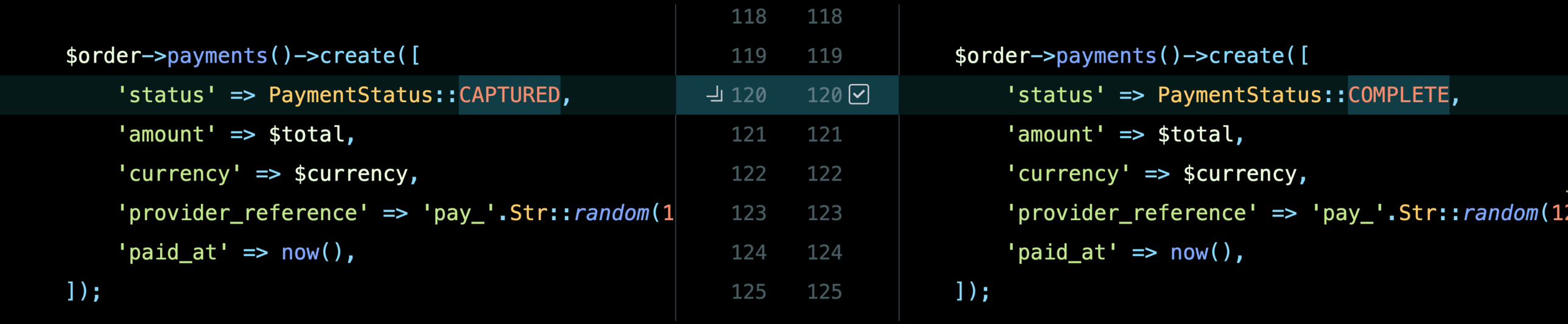
```php
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
        TicketStatus::ISSUED));
        $this->assertTrue(
            $order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null),
            'Every ticket in the order should have status of `issued`'
        );
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

```
sail@e99c11416379:/var/www/html$
```

Custom Constraints

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Every ticket in the order should have status of `issued`
Failed asserting that false is true.

/var/www/html/tests/BoxOfficeAssertions.php:25
/var/www/html/tests/Feature/BoxOfficeTest.php:114
```

Custom Constraints



Custom Constraints

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Failed asserting that two variables reference the same object.
--- Expected
+++ Actual
@@ @@
-App\PaymentStatus Enum #2957 (CAPTURED, 'captured')
+App\PaymentStatus Enum #2828 (COMPLETE, 'complete')
```

Custom Constraints

```php
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue(
            $order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null),
            'Every ticket in the order should have status of `issued`'
        );
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

Custom Constraints

```
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(
          PaymentStatus::CAPTURED,
          $order->payments->first()->status,
          "Confirmed orders should have a payment status of `captured`, but got `{$order->payments-
     >first()->status->value}`");
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue(
            $order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null),
            'Every ticket in the order should have status of `issued`'
        );
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

Custom Constraints

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Confirmed orders should have a payment status of `captured`, but got `complete`
Failed asserting that two variables reference the same object.
--- Expected
+++ Actual
@@ @@
-App\PaymentStatus Enum #2957 (CAPTURED, 'captured')
+App\PaymentStatus Enum #2828 (COMPLETE, 'complete')
```

Custom Constraints

```
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(
          PaymentStatus::CAPTURED,
          $order->payments->first()->status,
          "Confirmed orders should have a payment status of `captured`, but got `{$order->payments-
     >first()->status->value}`");
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue(
            $order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null),
            'Every ticket in the order should have status of `issued`'
        );
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

Custom Constraints

Custom Constraints

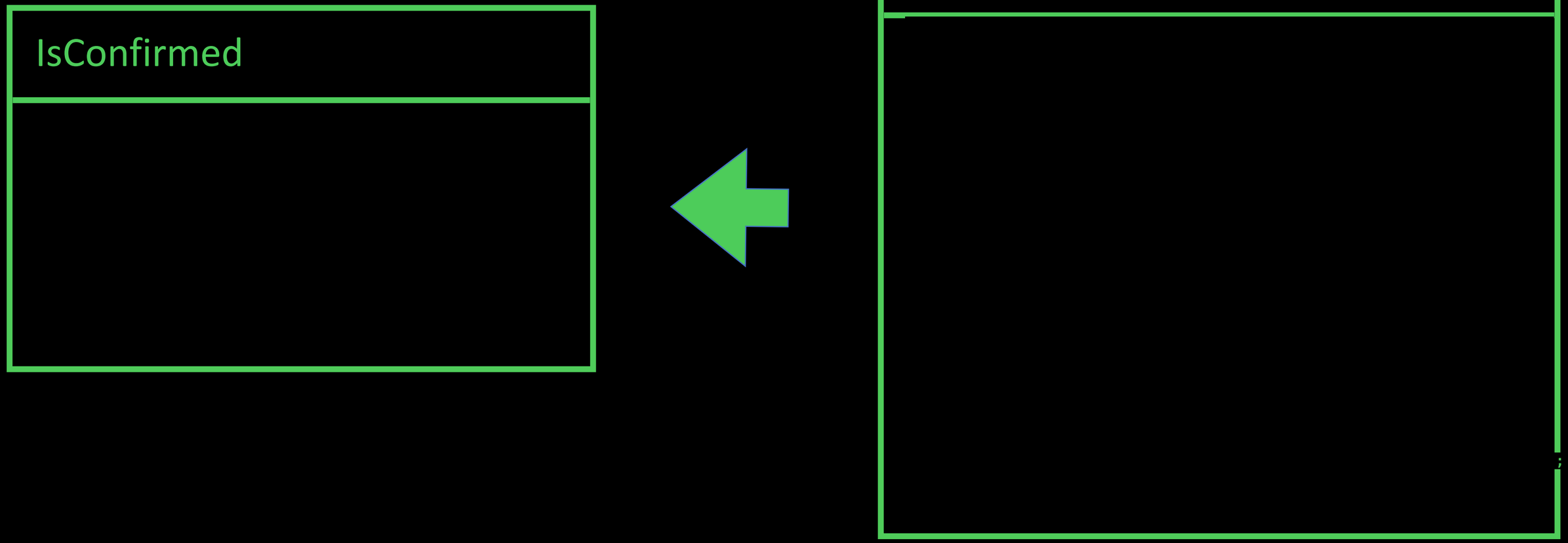
```php
<?php

namespace Tests\Constraints;

use PHPUnit\Framework\Constraint\Constraint;

class IsConfirmed extends Constraint
{









}
```

Custom Constraints

```php
use PHPUnit\Framework\Constraint\Constraint;

class IsConfirmed extends Constraint
{
    public function toString(): string
    {
        return 'is a confirmed order';
    }

    protected function matches(mixed $other): bool
    {
        return false;
    }

    protected function failureDescription(mixed $other): string
    {
        // ...
    }

    protected function additionalFailureDescription(mixed $other): string
    {
        // ...
    }
}
```

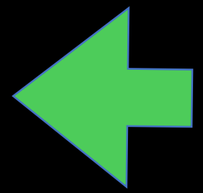
Custom Constraints

Custom Constraints



Custom Constraints

IsConfirmed



IsConfirmedTest

```
it_accepts_a_valid_confirmed_order();

it_rejects_an_order_with_an_invalid_status();

it_rejects_an_order_with_a_missing_confirmation_number();

it_rejects_an_order_with_a_missing_confirmed_at_timestamp();

it_rejects_an_order_with_an_unexpected_total();

it_rejects_an_order_without_a_payment();

it_rejects_an_order_with_an_unexpected_payment_status();

it_rejects_an_order_with_a_missing_payment_paid_at_timestamp();

it_rejects_an_order_without_any_tickets();

it_rejects_an_order_containing_a_ticket_with_an_invalid_status();

it_rejects_an_order_containing_a_ticket_with_a_missing_issued_at_timestamp();

it_rejects_an_order_containing_a_ticket_with_a_missing_code();
```

Custom Constraints

```php
<?php


class IsConfirmed extends Constraint
{
    //…


    public function toString(): string
    {
        return 'is a confirmed order';
    }

    protected function matches(mixed $other): bool
    {
        return $other instanceof Order && ! in_array(false, $this->checks($other), true);
    }

    //…
}
```

Custom Constraints

```php
class IsConfirmed extends Constraint
{
    //…

    protected function failureDescription(mixed $other): string
    {
        if (! $other instanceof Order) {
            return parent::failureDescription($other);
        }

        return "order {$other->confirmation_number} {$this->toString()}";
    }

    protected function additionalFailureDescription(mixed $other): string
    {
        if (! $other instanceof Order) {
            return '';
        }

        return collect($this->checks($other))
            ->map(fn (bool $passed, string $check): string => ($passed ? '  ✓ ' : '  ✗ ').$check)
            ->implode("\n");
    }
```

Custom Constraints

```php
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertSame(OrderStatus::CONFIRMED, $order->status);
        $this->assertNotNull($order->confirmation_number);
        $this->assertNotNull($order->confirmed_at);
        $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
        $this->assertCount(1, $order->payments);
        $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
        $this->assertNotNull($order->payments->first()->paid_at);
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->status ===
TicketStatus::ISSUED));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->issued_at !== null));
        $this->assertTrue($order->tickets->every(fn (Ticket $t): bool => $t->code !== null));
    }
}
```

Custom Constraints

```php
trait BoxOfficeAssertions
{
    public function assertOrderIsConfirmed(Order $order): void
    {
        $this->assertThat($order, new IsConfirmed);
    }
}
```

Custom Constraints

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Failed asserting that two variables reference the same object.
--- Expected
+++ Actual
@@ @@
-App\PaymentStatus Enum #2957 (CAPTURED, 'captured')
+App\PaymentStatus Enum #2828 (COMPLETE, 'complete')
```

Custom Constraints

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Failed asserting that order TH-AT6ETS is a confirmed order.
  ✓ status: confirmed
  ✓ confirmation number: TH-AT6ETS
  ✓ confirmed at: 2026-09-27 21:09:38
  ✓ total: 4500 = 4000 + 500
  ✓ payments: 1 (expected 1)
  ✗ payment status: complete
  ✓ payment paid at: 2026-09-27 21:09:38
  ✓ tickets: 2 (expected at least 1)
  ✓ tickets with status issued: 2 of 2
  ✓ tickets with issued at: 2 of 2
  ✓ tickets with code: 2 of 2

/var/www/html/tests/BoxOfficeAssertions.php:14
/var/www/html/tests/Feature/BoxOfficeTest.php:114
```

```
sail@e99c11416379:/var/www/html$
```

#ShipIt

Custom Comparators

A FEW
MOMENTS LATER

TicketHound is expanding to the **Canadian market**!

TICKETHOUND

+

Custom Comparators

```
> composer require brick/money
```

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertOrderIsConfirmed($order);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertCount(2, $order->tickets);
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Comparators

2013

Canada drops the penny.

Cash totals round to the nearest 5¢.

$0.01–0.02 → **$0.00**

$0.03–0.07 → **$0.05**

$0.08–0.09 → **$0.10**

Custom Comparators

Custom Comparators

> Payment Total ≠ Order Total

e.g.

> Order Total - $45.37
  > Digital - $45.37
  > Cash - $45.35

> Order Total - $45.38
  > Digital - $45.38
  > Cash - $45.40

Custom Comparators

Cash rounding

Some currencies do not allow the same increments for cash and cashless payments. For example, `CHF` (Swiss Franc) has 2 fraction digits and allows increments of 0.01 CHF, but Switzerland does not have coins of less than 5 cents, or 0.05 CHF.

You can deal with such monies using `CashContext`:

```php
use Brick\Money\Money;
use Brick\Money\Context\CashContext;
use Brick\Math\RoundingMode;

$money = Money::of(10, 'CHF', new CashContext(step: 5)); // CHF 10.00
$money->dividedBy(3, RoundingMode::Down); // CHF 3.30
$money->dividedBy(3, RoundingMode::Up); // CHF 3.35
```

Custom Comparators

> Add a `payment_method` field to the `payments` table

> Modify the the amount() method on our Payment object

```php
public function amount(): Money
{
    return Money::ofMinor(
        $this->amount,
        $this->currency,
        $this->method->context($this->currency)
    );
}
```

Custom Comparators

```
#[Test]
#[DataProvider('provideOrderTotals')]
public function canadian_cash_sales_are_rounded_to_the_nearest_nickel(int $price, int $total, int
$payment): void
{
    $event = Event::factory()->canadian()->create();
    $ga = TicketType::factory()->for($event)->currency('CAD')->price($price)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $event, new SeatRequest($ga, 1));

    $order = $this->boxOffice()->confirm($reservation, PaymentMethod::CASH);

    $this->assertEquals(Money::ofMinor($total, 'CAD'), $order->total());
    $this->assertEquals(Money::ofMinor($payment, 'CAD'), $order->payments->first()->amount());
}

public static function provideOrderTotals(): iterable
{
    yield 'Rounding Down' => [4287, 4537, 4535];
    yield 'Rounding Up' => [4288, 4538, 4540];
    yield 'Rounding Nowhere' => [4250, 4500, 4500];
}
```

Custom Comparators

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertOrderIsConfirmed($order);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());

    $this->assertCount(2, $order->tickets);
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

Custom Comparators

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertOrderIsConfirmed($order);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->payments->first()->amount());

    $this->assertCount(2, $order->tickets);
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

```
sail@5f355b55de49:/var/www/html$
```

Custom Comparators

```
4) Tests\Feature\BoxOfficeTest::canadian_cash_sales_are_rounded_to_the_nearest_nickel@Rounding Nowhere with data (4250, 4500, 4500)
Failed asserting that two objects are equal.
--- Expected
+++ Actual
@@ @@
 Brick\Money\Money Object (
     'amount' => Brick\Math\BigDecimal Object (...)
     'currency' => Brick\Money\Currency Object (...)
-    'context' => Brick\Money\Context\DefaultContext Object #2679 ()
+    'context' => Brick\Money\Context\CashContext Object #2664 (
+        'step' => 5,
+    )
 )
```

Custom Comparators

```
-- --
 Brick\Money\Money Object (
     'amount' => Brick\Math\BigDecimal Object (...)
     'currency' => Brick\Money\Currency Object (...)
-    'context' => Brick\Money\Context\DefaultContext Object #3002 ()
+    'context' => Brick\Money\Context\CashContext Object #487 (
+        'step' => 1,
+    )
 )
```

Custom Comparators

> The Problem…

```
$this->assertEquals(Money::ofMinor(4500, 'USD'), $order->payments->first()->amount());
```

Custom Comparators

```
$expectedMoney = Money::ofMinor(4500, 'USD');
$this->assertEquals($expectedMoney->getAmount(), $order->payments->first()->amount()->getAmount());
$this->assertEquals($expectedMoney->getCurrency(), $order->payments->first()->amount()->getCurrency());
```

Custom Comparators

```
$expectedMoney = Mone
$this->assertEquals($                                    ount()->getAmount());
$this->assertEquals($                                    amount()->getCurrency());
```

Custom Comparators

> A custom comparator controls how *assertEquals* and *assertNotEquals* compare objects of a specific type.

Custom Comparators

```php
use SebastianBergmann\Comparator\Comparator;

class MoneyComparator extends Comparator
{
    public function accepts(mixed $expected, mixed $actual): bool
    {
        // Can it compare the two values?
    }

    public function assertEquals(
        mixed $expected,
        mixed $actual,
        float $delta = 0.0,
        bool $canonicalize = false,
        bool $ignoreCase = false
    ): void {
        // Are the 2 values equal?
    }
}
```

Custom Comparators

```php
use SebastianBergmann\Comparator\Comparator;

class MoneyComparator extends Comparator
{
    public function accepts(mixed $expected, mixed $actual): bool
    {
        return $expected instanceof Money && $actual instanceof Money;
    }

    public function assertEquals(
        mixed $expected,
        mixed $actual,
        float $delta = 0.0,
        bool $canonicalize = false,
        bool $ignoreCase = false
    ): void {
        // Are the 2 values equal?
    }
}
```

Custom Comparators

```
    public function assertEquals(
        mixed $expected,
        mixed $actual,
        float $delta = 0.0,
        bool $canonicalize = false,
        bool $ignoreCase = false
    ): void {
        if (
            $expected->getAmount()->compareTo($actual->getAmount()) === 0 &&
            $expected->getCurrency()->getCurrencyCode() === $actual->getCurrency()->getCurrencyCode()
        ) {
            return;
        }

        throw new ComparisonFailure(
            $expected,
            $actual,
            $expected->getAmount() . ' / ' . $expected->getCurrency(),
            $actual->getAmount() . ' / ' . $actual->getCurrency(),
        );

    }
}
```

Custom Comparators

> Wire It Up…

```
class BoxOfficeTest extends TestCase
{
    protected Customer $customer;
    protected Event $event;

    protected function setUp(): void
    {
        parent::setUp();
        $this->customer = Customer::factory()->create();
        $this->event = Event::factory()->create();
   }
```

Custom Comparators

> Wire It Up…

```
use SebastianBergmann\Comparator\Factory;

class BoxOfficeTest extends TestCase
{
    protected Customer $customer;
    protected Event $event;
    protected MoneyComparator $comparator;

    protected function setUp(): void
    {
        parent::setUp();
        $this->customer = Customer::factory()->create();
        $this->event = Event::factory()->create();
        $this->comparator = new MoneyComparator();
        Factory::getInstance()->register($this->comparator);
    }

    protected function tearDown(): void
    {
        Factory::getInstance()->unregister($this->comparator);
        parent::tearDown();
    }
```

Custom Comparators

> Wire It Up…

```php
use SebastianBergmann\Comparator\Factory;

class BoxOfficeTest extends TestCase
{
    protected Customer $customer;
    protected Event $event;
    protected MoneyComparator $comparator;

    protected function setUp(): void
    {
        parent::setUp();
        $this->customer = Customer::factory()->create();
        $this->event = Event::factory()->create();
        $this->comparator = new MoneyComparator();
        Factory::getInstance()->register($this->comparator);
    }

    protected function tearDown(): void
    {
        Factory::getInstance()->unregister($this->comparator);
        parent::tearDown();
    }
```

```
sail@5f355b55de49:/var/www/html$
```

#ShipIt

Custom Object Exporters

Custom Object Exporters

```php
class CustomerOrdersTest extends TestCase
{
    #[Test]
    public function upcoming_orders_are_listed_soonest_first(): void
    {
        $past = $this->orderFor(‘Longhorn PHP 2025', ‘2025-10-23');
        $later = $this->orderFor('Google I/O 2027’, '2027-05-15');
        $sooner = $this->orderFor(‘PHP Tek 2027’, '2027-04-27');

        $this->assertEquals([$sooner, $later], $this->customer->upcomingOrders()->all());
    }
```

```
sail@5f355b55de49:/var/www/html$ ./vendor/bin/phpunit tests/Feature/CustomerOrdersTest.php
PHPUnit 13.3.2 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.5.8
Configuration: /var/www/html/phpunit.xml

F                                                                   1 / 1 (100%)

Time: 00:00.248, Memory: 56.50 MB

There was 1 failure:

1) Tests\Feature\CustomerOrdersTest::upcoming_orders_are_listed_soonest_first
Failed asserting that two arrays are equal.
--- Expected
+++ Actual
@@ @@
         'wasRecentlyCreated' => false
         'escapeWhenCastingToString' => false
         'attributes' => Array (
-            'id' => 3
+            'id' => 2
             'customer_id' => 1
-            'reservation_id' => 3
+            'reservation_id' => 2
             'status' => 'confirmed'
-            'confirmation_number' => 'TH-P2NDI0'
-            'subtotal_amount' => 5662
```

There’s Gotta Be a Better Way…

Custom Object Exporters

> A custom object exporter enables control over how objects are displayed on comparison failures

Custom Object Exporters

```php
class OrderExporter implements ObjectExporter
{
    public function handles(object $object): bool
    {
        // Can it export this $object?
    }

    public function export(
        object $object,
        Exporter $exporter,
        int $indentation,
        ExportContext $context
    ): string
    {
       // Export object to a string representation
   }
}
```

Custom Object Exporters

```php
class OrderExporter implements ObjectExporter
{
    public function handles(object $object): bool
    {
        return $object instanceof Order;
    }

    public function export(
        object $object,
        Exporter $exporter,
        int $indentation,
        ExportContext $context
    ): string
    {
        $event = $object->reservation->event;

        return sprintf(
            'Order %s (%s, %s)',
            $object->confirmation_number,
            $event->name,
            $event->starts_at->format('M j')
        );
    }
}
```

Custom Object Exporters

```php
use Tests\OrderExporter;

class CustomerOrdersTest extends TestCase
{
    protected Customer $customer;


    protected function setUp(): void
    {
        parent::setUp();
        $this->travelTo('2026-10-01');
        $this->customer = Customer::factory()->create();
        $this->registerObjectExporter(new OrderExporter());
     }
```

```
sail@94b212d639d7:/var/www/html$
```

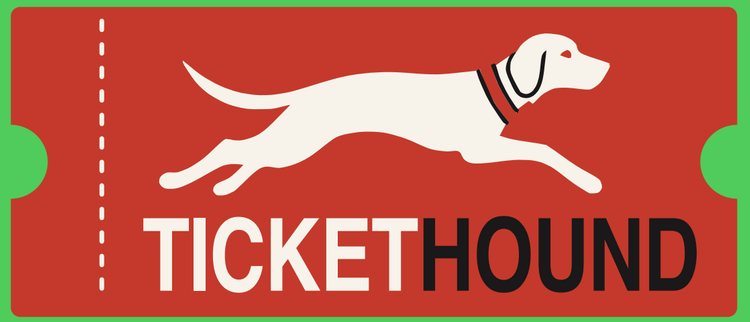
Custom Object Exporters

```
Failed asserting that two arrays are equal.
--- Expected
+++ Actual
@@ @@
 Array (
-    0 => Order TH-HGWERT (PHP Tek 2027, Apr 27)
-    1 => Order TH-JNZB9X (Google I/O 2027, May 15)
+    0 => Order TH-JNZB9X (Google I/O 2027, May 15)
+    1 => Order TH-HGWERT (PHP Tek 2027, Apr 27)
 )

/var/www/html/tests/Feature/CustomerOrdersTest.php:37

FAILURES!
Tests: 1, Assertions: 1, Failures: 1.
```

```
sail@5f355b55de49:/var/www/html$
```

#ShipIt

Let’s Recap…



Test Desiderata

> What Do You Want From Your Test Suite?

> Writable

tests should be cheap to write relative to the cost of the code being tested.

> Readable

tests should be comprehensible for reader, invoking the motivation for writing this particular test.

> Behavio(u)ral

tests should be sensitive to changes in the behavior of the code under test. If the behavior changes, the test result should change.

> Specific

if a test fails, the cause of the failure should be obvious.

Test Setup

> Visually Group Test Phases

> Avoid Red Herrings

> Centralize Shared Setup

Custom Assertions

> Identify Patterns of Repeated, Generic Assertions

> Assertions Specific To The Test Should Remain in the Test

Custom Constraints

> A More Wholistic Assertion

> Provide More Detailed Feedback to User

Custom Comparators

> Controls how *assertEquals* and *assertNotEquals* compare objects

> Useful for Working with Objects You Don’t Own

Custom Object Exporters

> Control How an Object is Presented on test failures

> Reduce the “Visual Noise” of a test failure

> More Direct Feedback to the User

PHPUnit: Beyond Assertions

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    Carbon::setTestNow('2026-09-08 19:00:00');

    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);

    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $this->assertSame(ReservationStatus::HELD, $reservation->status);
    $this->assertSame('2026-09-08 19:10:00', $reservation->expires_at->format('Y-m-d H:i:s'));
    $this->assertSame(2, $reservation->items->sum('quantity'));
    $this->assertEquals(Money::ofMinor(2000, 'USD'), $reservation->items->first()->price());
}
```

PHPUnit: Beyond Assertions

```php
#[Test]
public function holding_seats_creates_a_ten_minute_reservation(): void
{
    $ga = TicketType::factory()->for($this->event)->capacity(50)->create();

    $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $this->assertSame(48, $ga->remaining());
}
```

PHPUnit: Beyond Assertions

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $customer = Customer::factory()->create();
    $event = Event::factory()->create();
    $ga = TicketType::factory()
        ->for($event)
        ->price(2000)
        ->create(['capacity' => 50, 'name' => 'General Admission']);
    $reservation = $this->boxOffice()->hold($customer, $event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertSame(OrderStatus::CONFIRMED, $order->status);
    $this->assertNotNull($order->confirmation_number);
    $this->assertNotNull($order->confirmed_at);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertSame($order->subtotal_amount + $order->fee_amount, $order->total_amount);
    $this->assertCount(1, $order->payments);
    $this->assertSame(PaymentStatus::CAPTURED, $order->payments->first()->status);
    $this->assertSame($order->total_amount, $order->payments->first()->amount);
    $this->assertNotNull($order->payments->first()->paid_at);
    $this->assertCount(2, $order->tickets);
```

PHPUnit: Beyond Assertions

```php
#[Test]
public function confirming_a_hold_creates_a_paid_order_with_tickets(): void
{
    $ga = TicketType::factory()->for($this->event)->price(2000)->capacity(50)->create();
    $reservation = $this->boxOffice()->hold($this->customer, $this->event, new SeatRequest($ga, 2));

    $order = $this->boxOffice()->confirm($reservation);

    $this->assertOrderIsConfirmed($order);
    $this->assertEquals(Money::ofMinor(4000, 'USD'), $order->subtotal());
    $this->assertEquals(Money::ofMinor(500, 'USD'), $order->fees());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->total());
    $this->assertEquals(Money::ofMinor(4500, 'USD'), $order->payments->first()->amount());
    $this->assertCount(2, $order->tickets);
    $this->assertSame(ReservationStatus::CONVERTED, $reservation->fresh()->status);
    $this->assertSame(48, $ga->remaining());
}
```

PHPUnit: Beyond Assertions

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Failed asserting that false is true.

/var/www/html/tests/BoxOfficeAssertions.php:25
/var/www/html/tests/Feature/BoxOfficeTest.php:114
```

PHPUnit: Beyond Assertions

```
1) Tests\Feature\BoxOfficeTest::confirming_a_hold_creates_a_paid_order_with_tickets
Failed asserting that order TH-AT6ETS is a confirmed order.
  ✓ status: confirmed
  ✓ confirmation number: TH-AT6ETS
  ✓ confirmed at: 2026-09-27 21:09:38
  ✓ total: 4500 = 4000 + 500
  ✓ payments: 1 (expected 1)
  ✗ payment status: complete
  ✓ payment paid at: 2026-09-27 21:09:38
  ✓ tickets: 2 (expected at least 1)
  ✓ tickets with status issued: 2 of 2
  ✓ tickets with issued at: 2 of 2
  ✓ tickets with code: 2 of 2

/var/www/html/tests/BoxOfficeAssertions.php:14
/var/www/html/tests/Feature/BoxOfficeTest.php:114
```

PHPUnit: Beyond Assertions

```
sail@5f355b55de49:/var/www/html$ ./vendor/bin/phpunit tests/Feature/CustomerOrdersTest.php
PHPUnit 13.3.2 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.5.8
Configuration: /var/www/html/phpunit.xml

F                                                                   1 / 1 (100%)

Time: 00:00.248, Memory: 56.50 MB

There was 1 failure:

1) Tests\Feature\CustomerOrdersTest::upcoming_orders_are_listed_soonest_first
Failed asserting that two arrays are equal.
--- Expected
+++ Actual
@@ @@
         'wasRecentlyCreated' => false
         'escapeWhenCastingToString' => false
         'attributes' => Array (
-            'id' => 3
+            'id' => 2
             'customer_id' => 1
-            'reservation_id' => 3
+            'reservation_id' => 2
             'status' => 'confirmed'
-            'confirmation_number' => 'TH-P2NDI0'
-            'subtotal_amount' => 5662
```

PHPUnit: Beyond Assertions

```
There was 1 failure:

1) Tests\Feature\CustomerOrdersTest::upcoming_orders_are_listed_soonest_first
Failed asserting that two arrays are equal.
--- Expected
+++ Actual
@@ @@
 Array (
-    0 => Order TH-DVELW1 (Longhorn PHP 2026, Oct 15)
-    1 => Order TH-GOV6XW (Google I/O 2026, May 15)
+    0 => Order TH-GOV6XW (Google I/O 2026, May 15)
+    1 => Order TH-DVELW1 (Longhorn PHP 2026, Oct 15)
 )

/var/www/html/tests/Feature/CustomerOrdersTest.php:37
```

Tests --About

> TESTS ARE CODE.

Treat Them Like It.

> TESTS ARE TOOLS.

Make Them Work For You.

BE THE CHANGE YOU WANT TO SEE IN YOUR TEST SUITE

@dansoppelsa

https://joind.in/talk/ec1aa

PHPUnit: Beyond Assertions

$ ECHO "THANK YOU"

Questions?